ReconForge Limitations

Honest documentation of what ReconForge does **not** do, where manual work is required, and operational caveats every operator must understand.

What ReconForge Does NOT Do

Not a Full Exploitation Framework

ReconForge is a **reconnaissance and assessment** framework. It discovers, enumerates, and identifies potential vulnerabilities — it does not exploit them. There is no post-exploitation, no payload generation, no shell handling, no pivoting.

- No Metasploit-style exploit execution
- No reverse shell generation or handling
- No privilege escalation execution (it identifies paths, not walks them)
- No post-exploitation data exfiltration
- No persistence mechanisms

The exploit candidates phase in the web module (phase 4: `exploit`) is limited to **detection** via sqlmap `--batch --level=1 --risk=1` and wpscan enumeration. It does not perform actual exploitation unless sqlmap auto-exploits a confirmed injection — and this phase is opt-in only.

Not a Vulnerability Scanner

ReconForge is not Nessus, Qualys, or OpenVAS. The distinction matters:

Capability	Vulnerability Scanner	ReconForge
CVE database matching	Yes (comprehensive)	Limited (via nuclei templates)
Authenticated scanning	Yes (agent-based)	No (external tool wrapping)
Compliance checking	Yes (CIS, STIG, PCI)	No
Patch level assessment	Yes	No
Configuration auditing	Yes	No
CVSS scoring	Yes	No (uses own severity/confidence)
Continuous monitoring	Yes	No (single-run assessments)

ReconForge wraps external tools (nuclei, nikto) that perform some vulnerability detection, but it is not a substitute for a dedicated vulnerability management platform.

Not a Replacement for Manual Testing

ReconForge automates the **repetitive, mechanical** parts of reconnaissance. It cannot replace an operator's judgment, creativity, or domain knowledge. Specifically, it cannot:

- Understand application business logic
- Chain vulnerabilities creatively
- Adapt to novel or unusual architectures
- Make risk decisions requiring business context
- Perform social engineering
- Test physical security controls
- Assess mobile applications
- Evaluate source code (SAST)

Not a Compliance Scanner

ReconForge does not check compliance with:

- PCI DSS
- HIPAA
- SOC 2
- NIST 800-53
- CIS Benchmarks
- OWASP ASVS (systematic)
- GDPR technical controls

It may incidentally discover compliance-relevant issues (e.g., weak TLS, missing security headers), but it is not designed for systematic compliance assessment.

Not a Network Mapper

ReconForge uses nmap for port scanning and service detection, but it is not a network mapping or asset discovery platform:

- No network topology mapping
- No VLAN discovery
- No passive network monitoring
- No traffic analysis
- No asset inventory management
- No continuous discovery

The network module performs targeted scanning of specified hosts/CIDRs — it does not map entire network topologies.

Where Manual Validation is Required

Confirming Heuristic Findings

Any finding with `confidence: heuristic` is a pattern-based detection without concrete exploitation evidence. These **must** be manually validated. The FindingsManager enforces this by capping heuristic findings at `low` severity regardless of the requested severity.

Examples of heuristic findings:
- API endpoints inferred from URL patterns
- JWT weaknesses detected by header analysis alone
- Authorization issues based on response code patterns
- Technology detection based on HTTP headers or fingerprints

Exploiting Identified Vulnerabilities

ReconForge identifies potential vulnerabilities; exploitation is the operator's job:

- **SQL injection candidates** → Validate with manual sqlmap or Burp Suite
- **XSS candidates** → Craft and test payloads manually
- **SSRF indicators** → Test with out-of-band callbacks
- **IDOR/BOLA** → Verify with different user contexts
- **Kerberoastable accounts** → Request TGS and crack with hashcat
- **AS-REP roastable accounts** → Request AS-REP and crack offline

Validating Attack Paths

The AD module's `attack_paths.md` output describes theoretical attack chains. Each path must be manually validated:

- Verify prerequisites are actually met
- Test each step in sequence
- Confirm that mitigations haven't been applied since enumeration
- Account for network segmentation not visible to the scanner

Testing Business Logic Flaws

No automated tool can reliably test:

- Authentication bypass via workflow manipulation
- Price manipulation in e-commerce
- Race conditions in transaction handling
- Multi-step process abuse
- Referral/coupon abuse
- Account takeover via password reset flaws

Social Engineering

Completely out of scope:

- Phishing campaigns
- Pretexting calls
- Physical intrusion
- USB drop attacks
- Tailgating

Physical Security

Completely out of scope:

- Badge cloning
- Lock picking
- Dumpster diving

- Shoulder surfing

Where Heuristic Findings May Appear

ReconForge explicitly labels heuristic findings and caps their severity. Operators should know where these arise:

API Endpoint Discovery

The API discovery phase may infer endpoints from:
- URL patterns in responses
- JavaScript file references
- OpenAPI/Swagger spec parsing
- Common API path wordlists

These are educated guesses, not confirmed endpoints. Response codes (404 vs 200) provide some validation, but false positives occur (e.g., custom 404 pages returning 200).

JWT Analysis

The authentication phase's JWT analysis checks:
- Algorithm header (`alg: none` , HS256 vs RS256)
- Missing claims (`exp` , `iat` , `iss`)
- Signature presence

These are **structural** checks. They identify potential weaknesses but do not confirm exploitability. For example, detecting `alg: HS256` does not confirm the signing secret is weak — that requires offline cracking.

Authorization Fuzzing

The authorization phase uses pattern-based testing:
- Replaying requests with modified IDs
- Checking response code differences between roles
- Testing endpoint access without authentication

Response patterns are heuristic. A 200 response to an unauthorized request may be a generic error page, not actual data leakage.

Surface Intelligence

The surface module's correlation engine maps services to known attack vectors based on:
- Port/service associations
- Version-to-CVE lookups
- Service combination patterns (e.g., Kerberos + LDAP → AD)

These correlations are probabilistic. A service on port 8080 might not be HTTP. A service banner might be spoofed.

When Automation Should Stop

High-Risk Operations and OPSEC Modes

ReconForge enforces OPSEC boundaries through three modes:

Mode	Allowed Noise Levels	Use Case
`stealth`	`low` only	Red team, evasion testing, production environments
`normal`	`low`, `medium`	Standard penetration tests
`aggressive`	`low`, `medium`, `high`, `very_high`	CTF, lab, authorized full-scope tests

Each tool and technique has a noise rating in `core/detection_map.py`. The `OpsecChecker` blocks techniques that exceed the mode's threshold. When a technique is blocked, the log shows:

```
OPSEC BLOCKED: '<description>' not allowed in <mode> mode (noise: <level>)
```

You cannot override OPSEC from the CLI. To use a blocked technique, you must change the OPSEC mode. This is a deliberate safety rail.

Credential Brute-Forcing

Hydra brute-force testing is **opt-in only** via `--brute-force`:

```
python reconforge.py network --target 10.10.10.1 --brute-force
```

Without this flag, hydra is never invoked. Even with the flag, the tool configuration enforces safety limits:
- Maximum 4 concurrent tasks
- 3-second wait between attempts
- Maximum 10 attempts per account

These are defined in `config/tools.yaml` under `hydra.safety`.

Exploitation Attempts

The web module's exploit phase (phase 4) is opt-in. It only runs when:
1. `exploit` is explicitly included in `--phases`
2. The module receives `opt_in=True`

The authorization phase in the API module is similarly opt-in: it only runs when `authorization` is in `--phases`.

Destructive Testing

ReconForge does **not** perform destructive testing:
- No denial-of-service testing
- No data modification
- No account lockout (beyond hydra safety limits)

- No file upload exploitation
- No malware deployment

Tool Dependency Limitations

External Tools Required

ReconForge wraps external tools — it does not bundle them. If a tool is missing, the corresponding phase produces empty results and logs a `ToolNotFoundError`.

Required tools (framework will not start without these):
- `nmap` — Used by network, AD, and surface modules

Optional tools by module:

Module	Tool	Install Command
Network	enum4linux	`sudo apt install -y enum4linux`
Network	smbclient	`sudo apt install -y smbclient`
Network	ldapsearch (ldap-utils)	`sudo apt install -y ldap-utils`
Network	hydra	`sudo apt install -y hydra`
AD	enum4linux-ng	`pip install enum4linux-ng`
AD	impacket suite	`pip install impacket`
AD	bloodhound-python	`pip install bloodhound`
AD	netexec	`pipx install netexec`
Web	whatweb	`gem install whatweb`
Web	wafw00f	`pip install wafw00f`
Web	nikto	`apt install nikto`
Web	gobuster	`apt install gobuster`
Web	ffuf	`go install github.com/ffuf/ffuf@latest`
Web	wpscan	`gem install wpscan`
Web	nuclei	`go install github.com/projectdiscovery/nuclei/v2/cmd/nuclei@latest`
Web	sqlmap	`apt install sqlmap`
API	httpx	`go install github.com/projectdiscovery/httpx/cmd/httpx@latest`
API	arjun	`pip install arjun`

Tool Version Dependencies

ReconForge does not enforce specific tool versions. It passes arguments and parses output based on expected formats. If a tool updates its output format, parsers may break. Key version-sensitive tools:

- **enum4linux-ng** — Output format differs significantly from enum4linux (classic)
- **netexec** — Formerly CrackMapExec; binary name may be `nxc`, `netexec`, or `crackmapexec`
- **impacket** — Script names vary by install method (`GetADUsers.py` vs `impacket-GetADUsers`)
- **nuclei** — Template updates may change output structure

Platform-Specific Tools

- `nmap` SYN scans (`-sS`) and UDP scans require **root/sudo** privileges
- `bloodhound-python` requires network access to AD ports
- `wpscan` requires a Ruby runtime
- `whatweb` requires a Ruby runtime
- Go-based tools (ffuf, httpx, nuclei) require Go or prebuilt binaries

Optional Loot Encryption

The `--encrypt-loot` flag requires the `cryptography` Python package:

```
pip install cryptography
```

If not installed and `--encrypt-loot` is used, a warning is emitted and loot is stored in plaintext.

OPSEC Caveats

Stealth Mode Limitations

Stealth mode significantly reduces scan coverage:

- **Surface:** Only top 100 ports, T1 timing (very slow), no version detection
- **Network:** SYN scan only on curated port list, no script scanning, no UDP
- **AD:** Passive phase only — anonymous LDAP and null session SMB
- **Web:** Surface phase only — whatweb + wafw00f (technology fingerprinting)
- **API:** Discovery phase only — httpx probing and spec detection

Stealth mode is **not invisible**. Low-noise techniques still generate network traffic. IDS/IPS with behavioral analysis may still detect:
- Sequential port probes (even slow ones)
- SMB null session attempts
- LDAP anonymous bind attempts
- HTTP requests to known recon paths

Detection Risks by Module

Module	Stealth Risk	Normal Risk	Aggressive Risk
Surface	Low — slow SYN scan	Medium — version probes	High — full port scan
Network	Low — limited ports	Medium — service enum	High — scripts, UDP
AD	Low — anonymous only	Medium — LDAP/SMB queries	Very High — Blood-hound, RID cycling
Web	Low — passive finger-print	Medium — directory brute-force	Very High — nikto, sqlmap
API	Low — HTTP probes	Medium — endpoint fuzzing	High — authorization testing

Noise Generation

Tools that generate the most noise (highest detection risk):

1. **sqlmap** (`very_high`) — Active SQL injection testing
2. **hydra** (`very_high`) — Credential brute-forcing
3. **nikto** (`high` / `very_high`) — Comprehensive vulnerability scanning
4. **wpscan aggressive** (`very_high`) — WordPress plugin detection
5. **bloodhound full collection** (`very_high`) — AD graph enumeration
6. **nmap aggressive** (`very_high`) — OS detection + scripts + version

IDS/IPS Considerations

- **Signature-based IDS** will alert on nmap scans, nikto probes, and known tool User-Agents
- **Anomaly-based IDS** may detect the volume of requests from content enumeration (gobuster, ffuf)
- **WAFs** may block or rate-limit web fuzzing tools
- **AD monitoring** (e.g., Microsoft ATA/Defender for Identity) will detect Kerberoasting, AS-REP roasting, and Bloodhound collection
- **SIEM correlation** may link multiple tool signatures to a single source IP

Framework Scope

Reconnaissance and Assessment Focus

ReconForge is designed for the **reconnaissance and enumeration** phases of a penetration test. It maps to the first two stages of most kill chain models:

1. ✅ **Reconnaissance** — Target discovery, service enumeration, technology fingerprinting
2. ✅ **Scanning/Enumeration** — Port scanning, vulnerability identification, credential discovery
3. ❌ **Exploitation** — Manual operator responsibility

4. ❌ **Post-Exploitation** — Out of scope
5. ❌ **Reporting** — Generates raw findings, not polished client reports

Not a Replacement for Manual Exploitation

Findings from ReconForge are **starting points**, not conclusions. An identified Kerberoastable account is an opportunity — not a compromised account. An open admin panel is a lead — not a confirmed access.

Authorized Testing Only

ReconForge is designed exclusively for **authorized security assessments**. Every module generates network traffic that is visible to the target. Running ReconForge against unauthorized targets is illegal in most jurisdictions.

The `--engagement` and `--client` flags in workflow mode exist to maintain audit trails:

```
python reconforge.py workflow --target 10.10.10.1 \
    --engagement "Q1 Pentest" --client "Acme Corp" --operator "J. Smith"
```

Requires Operator Judgment

ReconForge does not make risk decisions. It presents findings with severity, confidence, and evidence. The operator must:

- Decide which findings to pursue
- Assess business impact
- Prioritize exploitation targets
- Choose OPSEC trade-offs
- Know when to stop
- Report results appropriately